

Details of Eternity II Solver eii

Louis Verhaard, January 22, 2009, version 2

This document describes the most important internal workings of eii, the Eternity II solver that could be downloaded from www.fingerboys.se/eii, and that also was incorporated in the distributed eternity2.fr solver (in versions 4.0 and 4.1). The eii solver does not primarily attempt to solve the whole Eternity II puzzle, but concentrates on the smaller prize; the Eternity II rules state "If no outright winner is found, Tomy may, at its discretion, announce a lesser prize for the highest-scoring entry since launch." (see www.eternityii.com). Rumors told that this smaller prize would be \$10 000, which turned later on turned out to be true.

I wrote this document because I got many questions from people that wondered how the solver works; I promised them that after 31/12/2008 I would write the details.

This paper is mainly written for people who know the rules of the Eternity II puzzle, how the scoring works, and have a little insight in how simple backtracking solvers work. Some references are made to the yahoo newsgroup; this was the most important medium for getting information or sharing ideas. This group is found on http://games.groups.yahoo.com/group/eternity_two Tons of information and ideas can be found here.

History

In September 2007 I had a lunch with my colleague Marcel Tünnissen, who told me about the Eternity II puzzle. He told also me that there had been an Eternity I puzzle, of which I never had heard, and that someone actually solved that puzzle and won 1 million pound. The next day I bought the puzzle and I did my best to win the 2 million dollars, but after a few months of work I gave up on that goal. But I found the puzzle still fascinating, so I continued to work on the puzzle to understand it better. In May 2008 I wanted to stop completely, but I changed my mind and decided to try to win the small prize. It was actually much more fun to find as many edges as possible, partly because it turned out that here it paid off to be smart, and also because of the competition element.

One of the things that frustrated me with Eternity II was that after all the "brilliant" ideas that I had tested to solve the puzzle, spending months of spare time, the best result of all this work was a completely stupid program known as "scan-row backtracker" that can be programmed in 20 lines of code or so. And other people seemed to experience the same.

During 2007, Dave Clark ran the eternity2.net site, using a distributed solver via the BOINC project. When Dave Clark announced that he would stop the project, he reported: "The highest scores achieved by eternity2.net users are in the mid 460ish range. (/480)". Hundreds of computers had participated in this project. At the same time, also the eternity2.fr site was around, with around 100 active users. I secretly joined this site, and estimated that this site had a capability of reaching a 465 or 466 score. These projects were really stimulating because I felt that these programs were not optimally tuned and I thought it would be really cool if I would be able to beat all these hundreds of computers using my two computers.

The basis for the eii solver is the mentioned stupid backtracker, but enhanced with some details that will be explained in the next chapters.

Note: to measure the strength of the different versions of the solver, I use the frequency at which solutions with a score of 463 are found, from now on called "463's".

Heuristics

Good Groups/Bad Groups

The most important factor in eii that contributes to finding good scores is the use of heuristics. The main idea is that we can obtain higher scores by saving pieces that fit well together until the end. There are endless many ways in

which this can be done. Several mechanisms have been proposed on the yahoo newsgroup.

One simple method is to assign a number to each individual piece, for example we could count all possible 2x2 piece configurations, and for every piece count how often it is part of such a configuration. Then we could try to use pieces with a low score at the beginning and save pieces with a high score until the end. However, it turns out that this does not work very well; just because a piece has a high score does not mean that it fit well with other pieces that are left. I did only very limited experiments with this, but did not get major improvements.

A much better approach is to try to create uneven distributions of colors, for example by getting rid of one colour early during the search and save other colours as long as possible. Max on the eternity newsgroup was able to solve a 4x49 puzzle, consisting of only inner pieces, using this approach (see message 5151). Without heuristics this would hardly have been possible. Originally I started with this approach, and got a big improvement compared to searching without heuristics at all. I got something like 10 times more often a 463 than without heuristics, and this could probably be further improved substantially.

But the heuristic that is used in the publicly released version of the solver is based on the tileability of groups of pieces, i.e. instead of assigning a value to one individual piece, we assign a value on a group of pieces, the higher the value (hopefully) the better these pieces fit together. The principle of considering groups instead of individual pieces was discussed by Brendan Owen in message 5058.

My idea was to divide all Eternity pieces into two groups, "good pieces" and "bad pieces". I actually announced this in message 5117:

My current plan is to investigate the following method: I use a backtracker with restarts (which currently restarts every 2e10 nodes). During one such small search, typically the first 80 or so nodes are left untouched, and pretty much from that point on the whole sub-tree is searched exhaustively.

Thus it seems to make sense to aim at a situation where these 170-180 remaining pieces that can be exhaustively searched tile as well as possible. A way to achieve this is to take a random group of maybe 180-190 pieces, use some method to determine the overall tileability of this group, and then gradually improve this group using by exchanging "bad pieces" with better pieces using some appealing/annealing method until some local optimum is reached that seems to be good enough to spend a few minutes on. From this "good group" we take the 10-20 "worst performers" and try to find a solution of the initial 80 pieces using as much as possible from the "loser group" and (as few as possible) "worst good ones" and use this 80-piece solution as the initial basis for one small search.

At this point I have no idea what can be expected, but I really hope that this or some other heuristic will give dramatically better results compared to searching randomly.

I contemplated over what method to use to determine the tileability of a group. One method that I considered was to try to use up all pieces in a square-like form using many small restarts for some limited amount of time and check the most visited depth; the higher, the better the tileability of the group.

I think that this method would have worked quite well, but I did not really like that my tileability measure would be dependent on luck, and, more importantly, I was lazy, so in the end I used a simpler method. I use as measure for the tileability of a group the number of solutions that this group can fill a 2x3 square containing 2 border squares, for example A2 A3 B2 B3 C2 C3 (corner pieces are ignored for a while).

I calculated "good groups" by using a separate program that uses a very simple algorithm: start off with a random good group, determine its tileability, and repeatedly exchange some random piece from the bad group with some piece from the good group; if the tileability improves, the pieces are really switched, otherwise the switch is undone. The algorithm stops if the tileability has not improved over some number of iterations, and the good group is saved. I ran this program for several days and obtained thus many good groups; the 60 groups with the best score were used in the solver.

For each of these 60 groups, also the best fitting corner pieces were determined, by calculating the number of solutions to a 3x3 square in the corner only pieces from the group. In the end, many of these 60 groups resembled each other, and not surprisingly they tend to contain very uneven colour distributions.

Why I chose to have 60 groups and not one? Partly I did not really want to rely on one single group; what if some other group would give significantly better results? Using more groups spreads the risk. And partly I was afraid that using only 1 group would unnecessarily limit the number of ways we get past the first part of the search: since the heuristics only "let through" a tiny fraction of all possibilities, I was a bit afraid for the risk that the same positions that pass the first stage are searched over and over again in different iterations. I thought anyway that probably this risk in reality would be negligible, but afterwards it may well be that I should have used many more groups. With many users, many, many possibilities are searched through, and every day the risk gets higher that the positions that are investigated have already been investigated before, for example by some other user. In January 2009 two eii users (jagbrain and Emmanuel) reported that they had gotten the same position on two different cores, and at that time I also got a duplicated position. *Note: this may very well be a serious obstacle in getting to 468 with the released version of eii. Therefore it may be necessary to loosen the heuristics, so that more positions are let through, and/or more groups should be added.*

I did some non-scientific measurements with these groups; I did one test where another set of groups were used that had a wide spread in their tileability score, and let the solver run a couple of days. The groups with high scores performed on average much better than groups with lower scores. Furthermore I monitored the results of the selected groups, and the 5 groups that had worst results were exchanged by 5 new groups before I publicly released the solver (but the 5 worst performing ones were not those with the lowest tileability score).

A good thing with this measurement for tileability is that we can give a value to each individual piece by counting in how many 2x3 solutions each piece occurs. This way, good pieces are divided into two sub-groups: "precious pieces" and normal "good pieces". The bad pieces are also divided into two sub-groups: "useless pieces" and normal "bad pieces".

Here is a typical picture of how these types of pieces could occur in a "solution" (score 463 or better). Note: eii start searching row by row from the bottom. '+' denotes a precious piece, '.' a good piece, 'b' a bad piece, 'U' a useless piece, 'M' the mandatory piece 139.

```
+ . + . b . b + + b b + . . + +
b . b b . b . . . b + + b b . b
. . . . . b . . + b + . . . + .
+ . . + . . . . + + + . . . + +
+ . . . b b + . . + . . . + + +
+ + b + + + + . . + . . + + . +
. . b + + . . . + + . b . . . .
. . . + . . . b + . . b . + . b
. + + . . b + M + + . . + + + b
U b . . + b . . + . . + . . . b
. . . . . . . . b b . . . . . b
b . b b b . . b b b . . b b . b
b b b b b b b b U U U b b b b b
b b U b U b b b b U b b b b b b
b b U U b b b b b U U b U b U U
b b U U b U b U b b b U b b U b
```

Unfortunately I had no good way to determine the optimal division into precious/good/bad/useless pieces; perhaps it is possible to build a model that predicts the effects of classifying pieces in this way, but I did not even attempt to build such model. Instead, tuning was done by observing the results after a few days; if the solver seemed to find more 463's than previously, I adopted the change. This is a very unreliable way of tuning, and I suspect that substantial improvements are possible to the publicly released version of eii.

The solver uses these groups in the following way: at every restart of a search, one of the 60 good groups is chosen randomly. Then the search is started, and at several predetermined depths some specific checks are made. For example until depth 63 is it forbidden to use any good piece, until depth 79 at most 6 good pieces may have been used, until depth 96 all useless pieces must have been used, until depth 100 is it forbidden to use any precious piece, etc. Also the checks made at these check-points are candidates for improvement as they are tuned in much the same way.

The use of these group based heuristics gave an enormous improvement over searching without heuristics, something like a factor 100 compared to searching without heuristics.

Heuristics for restarting a search

Other heuristics that are used by the eii solver are regarding restarts of searches. We want to prevent that we are stuck in a search that returns worse than average results. Such searches should be aborted as early as possible. And on the other hand, if we are in a "rush" and find many good solutions, we should not abort the search, because it is likely that continuing will give better results than restarting.

Initially I restarted every 20×10^{10} nodes and collected every 10^{10} nodes statistics about the number of found 463 solutions. It turns out that there is a clear correlation between the number of 463's found during the previous 10^{10} nodes and the number of 463's that can be expected during the next 10^{10} nodes (the more solutions we found last period, the more solutions we can expect next period); this can be exploited in an improved restart strategy.

Also an idea from max at the eternity yahoo group (see message 5876) was implemented: during the search it is measured how often each depth is visited. Also here I collected statistics, and indeed, the higher the most visited depth, the higher the 463 frequency. Using these statistics it was possible to determine the optimum visited depth at which the search should be aborted.

In the publicly released version of the solver, the search is aborted when any of the below conditions are met:

- we do not get past depth 128 within 200 million nodes. Probably we are stuck in the beginning of the search, perhaps it is impossible to combine the bad pieces/good pieces in such a way that we "get through"
- we have not found a single 463 within 4×10^9 nodes
- during the search it is measured how often each depth is visited. At regular points in the search the most visited depth is checked; if this is less than 165, the search is aborted.
- at every 10×10^9 nodes the number of found 463's is checked; if less than 40 463's were found, the search is aborted.

The effect is that ineffective searches are abandoned quite early, while effective searches ("rushes") can go on for hours and sometimes days. This heuristic proved to be very successful; something like a factor 5 better compared to a restart strategy that restarts after a fixed amount of nodes.

Usage of Hint Pieces

I have received quite many questions why the solver does not support hint pieces. Unfortunately, using hint pieces would only be a hindrance in the puzzle. I hope that the above explanation explains more clearly why: early during the search, the heuristics are very hard concentrated to getting rid of as many "useless" and "bad" pieces as possible; this would become harder by using hint pieces. The hint pieces may help a little bit if you try to find a complete solution, but this is not at all the objective of my solver, so they are completely irrelevant.

Search Algorithm

Another important part of the solver is the use of the search algorithm. Three aspects (apart from the heuristics) are most important when trying to find a good score:

1. the search order (the order in which squares are filled)
2. the algorithm used to "slip" edges

3. the speed of the solver

3) is the least important, but the easiest to explain: my solver is programmed in C, and uses similar techniques as discussed in message 3131 by "Mike", hamster_nz on the yahoo groups (but developed independently). My solver reaches a little bit over 50 million nodes/second on a 2.1 GHz laptop using "plain search" (no slipping), and roughly 30 million nodes/second when slipping edges. There are solvers that are much faster, so here my solver is not too bad, but nothing end-of-the-line.

1) and 2) are much harder to explain in a lucid way. These two aspects are dependent on each other, and also on the used heuristics and the score you want to reach.

I will try to describe how I selected these in eii.

Edge Slipping

With "edge slipping" is meant: "putting a piece on the board that does not match the colour of one or more of its neighbours". Of course, edge slipping is not interesting if you try to solve the whole puzzle, but it is necessary if you want to reach a high score.

First, for 2) I use the following method: edge slipping is totally integrated in the search. This is opposed to the method that is used in some solvers to employ different stages, for example a first stage where no edges are slipped, and a second stage where a good score from the first stage is used as the basis for an edge slipping method. I actually did a small experiment with this approach but got worse results than a "one phase" search where edge slipping is integrated.

In the solver a "slip array" is used that for any depth in the tree tells how many edges may be slipped at that depth. At any point in the search tree, we first try to place perfectly matching pieces. If the number of slipped edges so far is less than the slip array value at that depth, we will also try all so called "half-fitting pieces", pieces that fit except for 1 edge.

Edges between border pieces are never slipped, because border pieces tile better than inner pieces, so my intuition was that we should reserve the right to slip edges to inner pieces. But perhaps it would have been more optimal to also allow slipping of border edges at the very end of the search.

Perhaps/probably there are better ways to slip edges; I have not done much research in this area because of limited spare time. But given the used method of slipping edges, the question is: how should we configure the slip array?

Determining the Search Order and Slip Array

First of all, the perfect search order is dependent on the used heuristics as they affect the probability at which two pieces match. Because of this, I have not directly used any of the theoretical results for edge matching probabilities from the excellent work of Brendan Owen that have been published on the yahoo newsgroup (see for example message 5275).

Instead I used an iterative approach to find the best search order and slip array. Initially I just started with a simple scan-row search, and some slip array that starts slipping edges at quite low depths. At any depth I collected the following statistics:

- how often each depth is visited
- how often a perfectly fitting piece was placed at each depth
- how often a half-fitting piece was placed at each depth

From these statistics we can determine the probability at any depth that a piece fits perfectly and that a piece fits "half". So we let this program run over the night and the result is a nice collection of statistics.

Now the basic assumption is that the edge matching probabilities at some given depth are mostly affected by the used heuristics, and do not vary significantly due to the slip array or the search order. This makes it possible to write a program that given:

- the perfect-fitting and half-fitting edge matching probabilities (based on remaining number of border pieces/inner pieces)
- a search order
- a slip array

can estimate how often we will find a given target score (in terms of solutions/node).

And now we can feed this program with many different search orders and slip arrays, and of course we select the search order and slip arrays that gives the highest solution frequency. By caching intermediate results, this program is quite fast and can evaluate thousands of different input combinations per minute.

For example my initial estimates in order to find 465 without heuristics, the best square order I could find was

[P1 - P16 -- F1 - F16] [E1 - E3 -- A1 - A3] [E4 - A4 -- E12 - A12] E13-E16 D13 - D16 C13 - A13 B14 - B16 A14 - A16

and slip array { 188, 1, 196, 2, 203, 3, 208, 4, 213, 5, 218, 6, 222, 7, 225, 8, 228, 9, 231, 10, 234, 11, 237, 12, 239, 13, 241, 14, 243, 15 }

(i.e. at depths 188-195, at most 1 edge may be slipped, at depths 196-202, 2 edges may be slipped, etc.

The expected nodes/solution ratio for this search order/slip array combination was 5.93E13 per 465, more than 3 times better than the best slip array that could be found for a scan row search order.

When finding the best slip array for a certain search order, we exploited the fact that the most optimal slip array for some target score is usually very similar to the most optimal slip array for a slightly better target score. Example: suppose we find that the most optimal slip array to achieve a 467 score is { 192, 208, }. Then the most optimal slip array to achieve a 468 score will very likely start with { 192, 208, }, so we can limit our search for the best slip array for 471 to slight variations on the best slip array for score 470.

This calculation of the best search order/slip array should be redone when we aim for a new score, and also when significant improvements in heuristics make old edge matching probabilities obsolete. The nice thing with this is that improvements to heuristics give a double effect, the improvement in itself, and a (often little bit) more optimal slip array.

Resulting Search Order and Slip Array

It turns out that the optimal search order depends on the score you want to achieve (apart from the heuristics, as already discussed). For example if you want to go for 480, a scan row is quite optimal, but it is far from optimal if you want to achieve a score of 468 or lower. In the public solver (tuned for 468), the following square order is used:

[P1 - P16 -- E1 - E16] [D1 - D3 -- A1 - A3] [D4 - A4 -- D12 - A12] D13 - D16 C13 - A13 C14 - C16 B14 A14 B15 B16 A15 A16

and the following slip array is used:

{ 193, 1, 202, 2, 209, 3, 214, 4, 218, 5, 222, 6, 226, 7, 229, 8, 232, 9, 235, 10, 238, 11, 240, 12 }

Milestones

When I first started to try to find high scores I had misinterpreted the rules and thought that it was only allowed to place perfectly matching pieces or no piece at all. I found many solutions with 248 pieces on the board, with a personal high score of 450 points. When I understood my mistake, initially I tried a quick-and-dirty Java algorithm on my database of 248-piece scores; using this solver I was able to find my first 463, in the beginning of May. And a few days later I found a 464, by letting this Java solver loose on a couple of thousand positions I had collected during various other experiments earlier during my exploration of the puzzle.

Then I implemented the discussed program that given collected statistics on edge matching probabilities, search order, and slip array could find the probability of finding a certain target score. The prediction was that this should be found in 20×10^9 nodes, i.e. a few days of calculating. Indeed, after 5 days calculating the first two 465's dropped in, on May 22nd. At this time no heuristics were used. This non-heuristic solver found on average 6 463's per CPU and hour.

Without heuristics, my estimate was that it would take 10^{15} nodes to find 466 (or pretty much the rest of the year using my computers), so I started experimenting with heuristics, based on creating as uneven colour distributions as possible. The first 466 was found after only 7 days and this was submitted to the Eternity II adjudicators on May 28th. However I was very lucky, at this point I only had found 6 465's. At this point, the solver found around 30 463's per CPU and hour.

My estimates were that I had by now probably reached the same score or perhaps 1 point better than the eternity2.net and eternity2.fr distributed solvers, so it felt quite good. However since 466 turned out to be very doable, it seemed likely that I was not the first to reach this score, so in order to win, improvements were necessary.

I retuned all parameters to find 467, but at that point it was highly uncertain whether I could achieve this score during 2008. But gradually the heuristics improved. I started to use simplified group-based heuristics (without "useless"/"precious" subdivision) and the restart heuristics were added. Later on, the mentioned subdivisions with "useless" and "precious" pieces were added, and the solver got more and more efficient. I left my computers working 24x7. My estimate was that I would need to find on average 50 466 solutions per 467, perhaps more realistic 100 before the first could be expected. The heuristics were gradually improved, and 466 solutions dropped in, on average 2-3 each day. And finally, after having found more than 100 466's, and 2 millions of 463's, I found my first 467 in the middle of August. This solution was submitted with a request for "proof of delivery" (but this proof never came). Shortly after I found 3 more 467 scores. The solver found at this time around 700 463's per CPU and hour.

I was now quite sure that I had surpassed both distributed solvers. I estimated that 467 should probably be enough to win the small prize, but I thought that most likely there would be a few people in the world that have implemented similar or better solvers than I, perhaps with access to a lot more computer power, and that at least one of these people should have found a 467 earlier than me.

It would be an almost hopeless task to find 468 with my limited computer power at hand and time running out. I did one important improvement: previously I had for some obscure reason not considered corner pieces at all in good/bad groups, and not included border pieces in "precious" and "useless"; these were added and now the solver found 463's at a rate of 2000 per CPU and hour. Now I found more than 10 466 scores each day, but time was running out, and I would need to find on average 60 467's for each 468 (and 6×10^{15} nodes) with the current heuristics.

After this I was not able to improve the program anymore so I decided to make my program publicly available and rely on brute force to try to achieve 468. On the 22nd of September eii 1.0 was released, and it got many positive reactions. To even spread the program further I also started a cooperation with Antoine Jacquet ("royale") of eternity2.fr. By the end of 2008, at least 80 processors were running eii day and night, and 467 was found more than 50 times by the eii users, but unfortunately no 468 was found. But it was very nice mailing with eii/eternity2.fr users, so all the nice feedback was certainly worth the effort, even though the end result was disappointing! (*At least that's what I thought when the year was over*)

On the thirteenth of January, my wife Anna got an email with subject something like "congratulations! you have won 10 000\$!!". Of course she was convinced that this was spam and deleted the message. But a few seconds later she thought "wasn't that the amount you could win at Eternity II?" so she undeleted the mail and jumped off her chair when she realized it was really true! She called me and of course I was happy, but also very surprised to have won.

Then followed a period where we got a lot of attention: lots of people mailed, or congratulated on the yahoo newsgroup, two newspapers published articles with title like "family in Lund best in the world at puzzles" and our family appeared on a local television station! It was quite nice to be a little bit famous just once in a lifetime! Since it probably won't happen again, me and my family fully enjoyed this period. You can imagine how popular my 7-year

old daughter was at school, the day after she had been on tv! I even received phone calls from unknown people that wondered how they could buy the puzzle. My wife and I found it originally a bit strange why media were interested at all in something as unimportant as "tried to solve a puzzle but failed", but from all positive reactions we understood that this was a perfect fit for the "happy news" category. Always nice to spread some good news!

The future: getting 468 or higher

One thing that stroke me in the beginning was how well the theoretical estimates that I made beforehand fit with actually obtained results, often within 10-20% accuracy, in terms of frequency of solutions/nodes. But it seems that for high scores, 467 or higher, effects are playing a role that make the theoretical estimates too optimistic. My only indication are the empirical results for 467: in the publicly released version of the eii solver, according to my theoretical estimates 467 should be found once every 100×10^9 nodes. But this estimate turned out to be too optimistic, in reality they occurred more than 2 times more seldom. I suspect that 468 is even harder to find (compared to the theoretical estimate).

Perhaps the total number of 467 and 468 is so small that the use of heuristics becomes less useful; as Brendan also noted in message 5036: "Heuristics are generally only useful if there are multiple solutions, guiding your solver to find the easiest one."

Also it is likely that parity plays a role, as Max pointed out in message 6488. When you slip 1 edge of a color, you know that you have to slip at least one more edge later on with that same colour, so for every new colour you slip, you will have to pay later on with another slip. The end effect is that parity of slipped edges reduces the total number of positions with a very high score (like 468 or higher); this parity effect is not at all accounted for in my model.

Conclusions

The eii solver is not very interesting from a theoretical point of view; it is based on quite simple ideas and mathematics. Still its results are an order of magnitude better than any of the other publicly available solvers that have "high score finding" capabilities. I think that its relative success is mostly the result of solid engineering work. By starting with an ok implementation and collecting many statistics on different aspects that influence the efficiency of the solver, and by building simple and small models that could be used to select "optimal" configuration parameters (like slip array, restart configuration parameters), it was possible to gradually improve the program to steer it better and better into those parts of the search tree where we can expect high scores.

But still it should be possible to improve the program considerably; especially I expect that it is possible to improve the heuristics used for guiding the search. It would be exciting to learn what methods other people have used and what results they got.

The best current estimate is that there are only around 20 000 solutions (with score 480) in total of the Eternity II puzzle; the heuristics used in eii are not likely to help much to find one of these solutions.

Acknowledgements

First I would like to thank Marcel Tünnissen and Ramon van der Winkel for many lively and fruitful discussions. Thanks to Brendan Owen and Max, who have investigated many aspects of the Eternity II puzzle and shared their valuable findings on the yahoo newsgroup, giving me and many others a better insight in this beast. Thanks to Antoine Jacquet for the cooperation to embed eii into the distributed solver of eternity2.fr. Also thanks to all people who have tried the eii solver and the eternity2.fr solver. And finally I would like to thank Anna and my children: without them, this puzzle might have driven me insane...

Appendix: Details to determine Slip Array

Here are some of the details of the mentioned algorithm that I used to determine the optimum search order. To make it easy for myself I show some snippets of my Java class that does this job, even though the Java code is really quite

messy (it evolved over time, and was originally computing something else).

The original input of the algorithm is

- a search order
- a slip array
- statistics at every depth:
 - the probability that a piece matches
 - the probability that a piece fits "half"

As mentioned before these statistics are obtained by letting the program run for some time, instead also theoretical estimates could be used.

Output of the algorithm is an estimate (in terms of times/second) how often we will get to the "end", i.e. reach the target score set up by the slip array.

My algorithm does first some preparations and initializes an array called "ratios" of the below class.

```
private static class Ratios {
    int square;
    /** nr. of pieces left of the type that is to be put (4*nr pieces for inner squares) */
    int n;
    /** prob. that 1 random tile would fit perfectly */
    double fitProb;
    /** prob that 1 random tile would fit with 1 slipped edge */
    double halfFitProb;
}
```

Given these ratios and the slip array we can build a markov chain with state (depth, nr. slipped edges at that depth), with transitions with a certain probability to (depth+1, nr. slipped edges) and (if nr. slipped edges at that depth is less than the allowed number) a probability to (depth+1, nr. slipped edges + 1).

And using this chain you can work out the probability from going from the beginning until the end and the nodes you would have to search.

Here is some code:

```
private static LinkedList<Double> nrNodes(int depth, int skippedSquares) {
    LinkedList<Double> oldResult = savedResults.get(1000*depth + skippedSquares);
    if (oldResult != null) {
        return oldResult;
    }
    int square = SquareOrder.squareOrder[depth];
    if (square < 0) {
        LinkedList<Double> result = new LinkedList<Double>();
        result.add(1.0);
        return result;
    }
    int type = Board.nrBorderEdges(square);
    Ratios rat = ratios[depth];
    // simulate that we place a piece on square
    LinkedList<Double> list1 = nrNodes(depth + 1, skippedSquares);
    list1.addFirst(rat.n * rat.fitProb);
    LinkedList<Double> result = new LinkedList<Double>(list1);
    if (skippedSquares < slipArray[depth] && (type < 2)) {
        // simulate that we slip an edge
        LinkedList<Double> list2 = nrNodes(depth + 1, skippedSquares + 1);
        list2.addFirst(rat.n * rat.halfFitProb);
```

```

        result = mergeRatios(list1, list2);
    }
    savedResults.put(1000*depth + skippedSquares, new LinkedList<Double>(result));
    return result;
}

/**
 * Merges two lists of ratios into one combined list
 * @param ratios1 list of branch factor ratios (as result of fitting)
 * @param ratios2 list of branch factor ratios (as result of half-fitting)
 * @return
 */
private static LinkedList<Double> mergeRatios(List<Double> ratios1, List<Double> ratios2) {
    LinkedList<Double> result = new LinkedList<Double>();
    double visit1 = 1;
    double visit2 = 1;
    double total = 1;
    for (int i = 0; i < ratios1.size(); ++i) {
        double v1_new = visit1 * ratios1.get(i);
        double v2_new = visit2 * ratios2.get(i);
        result.addLast((v1_new + v2_new) / total);
        visit1 = v1_new;
        visit2 = v2_new;
        total = visit1 + visit2;
    }
    return result;
}

```

Here nrNodes returns a list of double, representing at each depth the branching factor of getting to the next depth. A cache ("savedResults") remembers all calculations, so the results can be computed efficiently despite the recursion. Method mergeRatios combines two lists of branching factors (one for perfectly fitting pieces, the other for half fitting pieces) into one combined list of branching factors.

By calling nrNodes(0, 0) we obtain the resulting list of branching factors at every depth, and from list this we can easily work out how many nodes will be required to get to put all pieces on the board (with slipping).